TECMILENIO

Máster:
Inteligencia Artificial

Tetramestre:
Septiembre-diciembre 2026

Materia:
Gestión de Proyectos de inteligencia Artificial

Proyecto Fase II · Manual de despliegue en la nube

Matrícula y nombre del estudiante:
T07175241 - Gabriel Alejandro García Ángeles

Fecha: 01/11/2026

1. Información general del documento

Tabla 1: Información general.

Elemento	Descripción
Nombre del proyecto	Chatbot de estatus de órdenes y fecha estimada de entrega (ETA) de Componentes Automotrices del Bajío (CAB)
Versión del documento	1.0 (corresponde a app-v1.0.0)
Fecha	01/11/2026
Responsable técnico	Gabriel Alejandro García Ángeles (T07175241)
Institución	Universidad Tecmilenio · Maestría en Inteligencia Artificial (CAB es una empresa ficticia)
Estado del proyecto	Prototipo validado en entorno local con contenedores; listo para desplegarse en un PaaS
Repositorio	https://github.com/gagarang25/ProyectoFase2
Modelo del router	https://huggingface.co/gagarang/router-intencion-cab-mdeberta (etiqueta model-v1.0.0)

2. Introducción

2.1 Propósito del documento

Este manual describe cómo se construye, se ejecuta y se despliega en la nube el prototipo del chatbot de CAB definido en la Fase 1 del proyecto. Su objetivo es que cualquier persona con conocimientos básicos de Docker pueda repetir el proceso a partir del repositorio y obtener el mismo resultado. La estructura sigue la plantilla de documentación técnica del Tema 16, alineada con la norma ISO/IEC 23053:2022, que propone describir un sistema de aprendizaje automático por sus componentes, sus funciones y su ciclo de vida completo, no solo por su código (International Organization for Standardization [ISO], 2022; SG Systems, 2025).

2.2 Alcance

El documento cubre:

- La arquitectura del sistema y la integración entre frontend, API y modelos.
- Los requerimientos técnicos y la estrategia de despliegue (modelo de implementación y de servicio).
- El proceso paso a paso: construcción de las imágenes, ejecución local, simulación del despliegue en la nube y despliegue en un PaaS (AWS Elastic Beanstalk, con Google Cloud Run y Azure Container Apps como alternativas).
- La verificación posterior al despliegue, la operación, el monitoreo y la solución de problemas.

No cubre el desarrollo y la evaluación detallada de los modelos, que están en el documento de validación y pruebas, ni la integración real con SAP S/4HANA, que en el prototipo se sustituye por datos simulados.

3. Marco normativo y referencias

- ISO/IEC 23053:2022, marco para sistemas de inteligencia artificial basados en aprendizaje automático (ISO, 2022).
- ISO/IEC 22989 (conceptos y terminología de IA) e ISO/IEC 23894 (gestión de riesgos de IA), como referencia de vocabulario y de la matriz de riesgos de la Fase 1.

- Ley Federal de Protección de Datos Personales en Posesión de los Particulares (Cámara de Diputados del H. Congreso de la Unión, 2025), que condiciona dónde se alojan los datos de los clientes.
- Documentación interna del proyecto: caso de negocio de la Fase 1, Actividades 4 a 6, README del repositorio y documento de validación y pruebas de la Fase 2.

4. Visión general del sistema de IA

4.1 Objetivo del sistema

Hoy un cliente B2B de CAB, normalmente el planeador de materiales de una armadora, manda un correo o llama a su ejecutivo para saber cómo va su orden y cuándo llegará. CAB recibe unas 4,500 consultas de este tipo al mes. El sistema permite hacer esa pregunta por escrito, en español o en inglés, y recibir en segundos el estatus y una fecha estimada con rango y motivos. Si el mensaje es una queja, una petición de hablar con una persona o algo que el modelo no entiende con seguridad, la conversación pasa a un ejecutivo.

4.2 Tipo de sistema de IA

Es un sistema híbrido que combina tres tipos de componentes:

- **Aprendizaje profundo (procesamiento de lenguaje natural):** un modelo de lenguaje tipo transformer, mDeBERTa-v3, ajustado para clasificar la intención del mensaje en cinco clases (He et al., 2023; Laurer et al., 2024).
- **Aprendizaje automático supervisado sobre datos tabulares:** una regresión que estima los días de entrega y un clasificador binario de riesgo de retraso, ambos de tipo gradient boosting.
- **Reglas de negocio:** la política de decisión (responder, preguntar, escalar o canalizar), la extracción del número de orden y las plantillas de respuesta.

4.3 Actores involucrados

- **Usuarios finales:** planeadores de materiales de los clientes del piloto (cuatro armadoras y un proveedor Tier 1).
- **Operadores del sistema:** ejecutivos de servicio al cliente, que reciben las conversaciones escaladas.
- **Administradores técnicos:** el área de TI y el especialista en MLOps, que despliegan, monitorean y actualizan los modelos.

5. Arquitectura del sistema de IA

5.1 Descripción general

La arquitectura separa el frontend del backend. El frontend es lo que ve el usuario en el navegador; el backend contiene la lógica de negocio, los modelos y los datos; y la API es el conjunto de reglas con el que ambos se comunican (Chen, 2025). Elegí una API REST porque usa HTTP y JSON, es fácil de probar y es el estilo más usado para este tipo de servicios (The Postman Team, 2023). La Figura 1 muestra el recorrido de un mensaje.

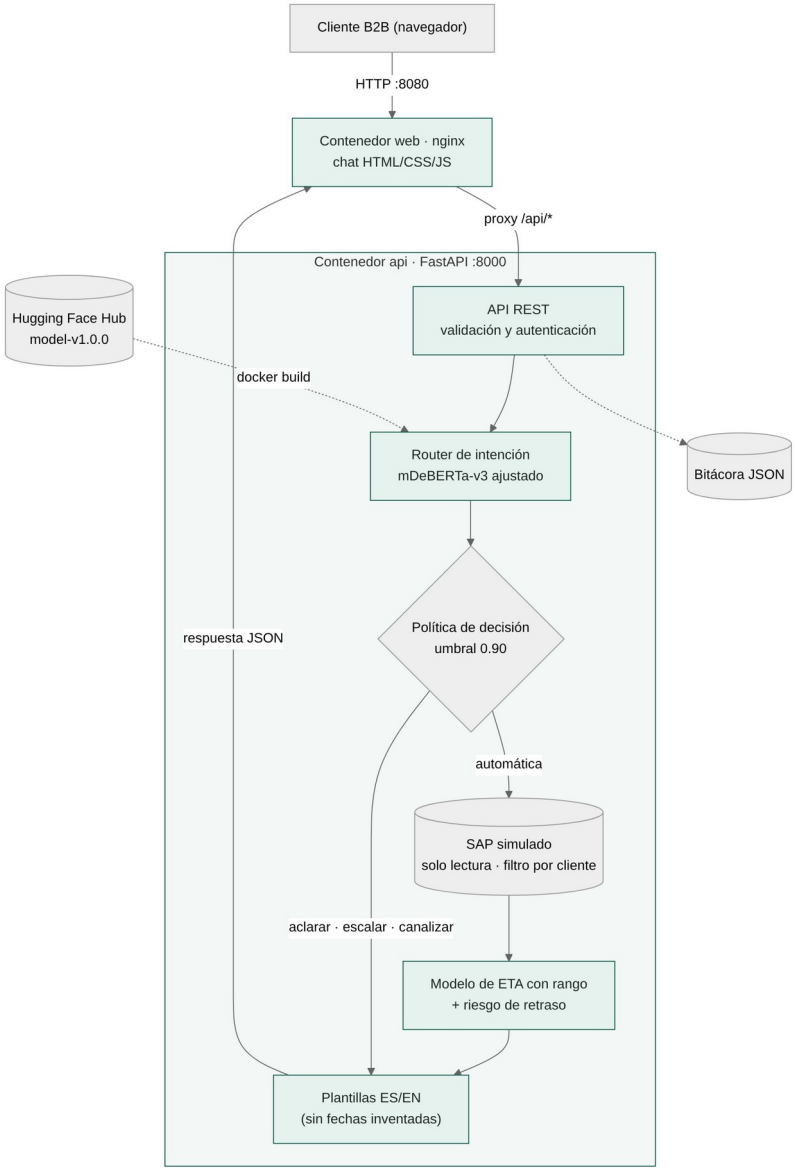


Figura 1: Arquitectura de la solución y recorrido de un mensaje.

El mensaje llega al contenedor web (nginx), que entrega la página del chat y reenvía las llamadas a /api al contenedor de la API. Ahí el router clasifica el mensaje y la política de decisión elige la acción. Si la acción es automática, la API consulta el SAP simulado, siempre filtrando por el cliente que inició sesión, y calcula la fecha estimada y el riesgo de retraso. La respuesta se arma con plantillas: el sistema nunca redacta una fecha por su cuenta, solo inserta las que vienen de SAP o de los modelos. Así se atienden los dos riesgos críticos de la Fase 1, la fuga de información entre clientes (R04) y la alucinación de estatus o fechas (R05).

5.2 Componentes del sistema (según ISO/IEC 23053)

Tabla 2: Componentes del sistema con la terminología de la plantilla.

Componente	Descripción en el proyecto
Fuente de datos	SAP simulado: 406 órdenes del piloto de 5 clientes al 28/10/2026 (datos de la A6). En producción, conexión de solo lectura a SAP S/4HANA.

Componente	Descripción en el proyecto
Procesamiento de datos	Validación de la solicitud (Pydantic), extracción del número de OC con una regla, detección de idioma y cálculo de variables derivadas (holgura).
Modelo ML	Router mDeBERTa-v3 ajustado (model-v1.0.0, Hugging Face Hub); modelo de ETA y clasificador de riesgo (model-v1.0.0, joblib).
Motor de inferencia	API FastAPI ejecutada con Uvicorn en CPU; carga los modelos una sola vez al arrancar.
Interfaz / API	API REST documentada en /docs (Swagger) y chat web en HTML, CSS y JavaScript servido por nginx.
Infraestructura	Dos contenedores Docker en local (docker compose); una sola imagen en el PaaS de la nube.

Tabla 3: Endpoints de la API.

Método	Ruta	Autenticación	Descripción
GET	/api/salud	No	Estado de la API y versiones de modelos, datos y código
POST	/api/intencion	No	Intención, confianza, probabilidades y acción del router
POST	/api/chat	Bearer	Flujo completo: router, SAP, ETA, riesgo y respuesta
GET	/api/ordenes	Bearer	Órdenes del cliente autenticado
GET	/api/ordenes/{oc}	Bearer	Estatus de una OC (404 si no es del cliente)
GET	/api/ordenes/{oc}/eta	Bearer	Fecha estimada, rango, motivos y riesgo
POST	/api/eta/estimar	No	ETA y riesgo para los datos de una orden

6. Datos en el entorno de despliegue

6.1 Tipos de datos

- **Datos de entrada:** el texto del mensaje (máximo 1,000 caracteres), el token del cliente, la OC de la conversación y, en el endpoint de simulación, las 22 variables de una orden (planta, transportista, carga de planta, aduana, etc.).
- **Datos de salida:** la respuesta en texto, la intención con su confianza, la acción, el estatus de la orden, la fecha estimada con su rango, los motivos y la probabilidad de retraso.
- **Datos derivados:** la holgura de la fecha comprometida, el idioma detectado, el folio de escalamiento y las advertencias cuando una entrada está fuera de lo que el modelo vio al entrenarse.

6.2 Gestión de datos

- **Validación de entrada:** FastAPI rechaza con código 422, antes de llegar a los modelos, los mensajes vacíos, demasiado largos o con valores imposibles.
- **Almacenamiento:** el prototipo no tiene base de datos; los datos de SAP se leen de un archivo dentro de la imagen y son de solo lectura.
- **Protección:** cada consulta se filtra por el cliente autenticado; una OC de otro cliente recibe la misma respuesta que una inexistente.
- **Retención:** la bitácora guarda una línea por conversación (cliente, intención, confianza, acción, OC, tiempo) y no guarda el texto del mensaje. La Fase 1 definió conservarla 12 meses (riesgo R09).

7. Modelos de machine learning en producción

7.1 Descripción de los modelos

Tabla 4: Modelos desplegados y su desempeño en prueba.

Modelo	Algoritmo	Entrada → salida	Desempeño (prueba)
Router de intención	mDeBERTa-v3 ajustado (279 millones de parámetros), calibrado con temperatura 0.59 y umbral 0.90	Texto → 5 intenciones con probabilidad	Exactitud 0.964 en 160 mensajes escritos a mano (español 0.956); latencia en CPU de 67 ms por mensaje
ETA	HistGradientBoosting (pérdida absoluta) + cuantiles 10 % y 90 %	22 variables de la orden → días de entrega con rango	MAE 1.05 días frente a 1.72 de la fecha de SAP; cobertura del rango 90.9 %
Riesgo de retraso	HistGradientBoosting (campeón de la A6), umbral 0.17	22 variables → probabilidad de retraso	AUC 0.818, recall 0.798 (igual que la A6)

7.2 Separación entre entrenamiento y operación

La norma ISO/IEC 23053 pide distinguir el entrenamiento de la operación. En este proyecto están separados por diseño. Los modelos se entrenan fuera de la API, con los scripts de la carpeta entrenamiento/, a partir de un commit de Git ya publicado: el router en GPU (cuaderno de Colab o una RTX 4060 local) y los modelos tabulares en CPU. Después se publican con la etiqueta model-v1.0.0. La API solo carga los modelos y hace inferencia.

7.3 Ciclo de vida

- **Versión actual:** model-v1.0.0 para los tres modelos (router entrenado en el commit 082d216; modelos tabulares en el mismo commit).
- **Actualización:** reentrenamiento trimestral del modelo de ETA y del de riesgo (Fase 1, riesgo R10) y del router cuando el piloto aporte conversaciones reales.
- **Criterios de reemplazo:** un modelo se reemplaza si dispara alguna de las alarmas del apartado 9.1 durante dos semanas, o si una versión nueva lo supera en la misma prueba, con las mismas condiciones y sin empeorar la precisión de las respuestas automáticas.

8. Proceso de despliegue del sistema

8.1 Estrategia de despliegue

Para decidir dónde y cómo desplegar usé las dos clasificaciones del Tema 16: el modelo de implementación (dónde vive la infraestructura) y el modelo de servicio (cuánto administra uno y cuánto el proveedor) (Martínez, 2024; China y Goodwin, s.f.).

Modelo de implementación. Para el prototipo, nube pública: los datos son simulados y no hay información confidencial. Para la operación real recomiendo una nube híbrida: SAP S/4HANA y los datos de los clientes se quedan en la red privada de CAB, y en la nube pública solo corre el contenedor de la API, que consulta SAP por una conexión segura de solo lectura. Así se aprovecha la escalabilidad de la nube sin sacar datos protegidos por la LFPDPPP ni información cubierta por las cláusulas de confidencialidad con las armadoras (riesgo R07).

Modelo de servicio. Elegí una API REST en contenedores sobre una plataforma como servicio (PaaS). Con IaaS tendría que administrar el servidor, el sistema operativo y la seguridad. Con FaaS cada función arranca bajo demanda y tendría que cargar un modelo de más de 500 MB en cada arranque en frío. SaaS no aplica porque la aplicación es propia. En un PaaS entrego la imagen y el

proveedor se encarga de los servidores, el HTTPS y el escalado, que es lo que necesita un equipo pequeño. La Tabla 5 compara las opciones evaluadas.

Tabla 5: Opciones de despliegue evaluadas.

Opción	Modelo	Ventajas	Limitaciones	Decisión
AWS Elastic Beanstalk (plataforma Docker)	PaaS	Es la plataforma del tutorial del Tema 16; construye la imagen desde el Dockerfile y crea la instancia, el dominio y las alarmas	Requiere cuenta de AWS; hay que elegir una instancia de al menos 4 GB (t3.medium)	Plataforma principal del manual
Google Cloud Run	PaaS de contenedores (serverless)	Escala a cero, cobra por uso, admite varios GB de memoria por instancia	Requiere cuenta de facturación; arranque en frío de varios segundos por la carga del modelo	Alternativa
Azure Container Apps	PaaS de contenedores	Escalado automático y red virtual privada, útil para la nube híbrida con SAP	Requiere suscripción	Alternativa
Hugging Face Spaces (Docker)	PaaS de contenedores	Mismo ecosistema que el modelo; construye desde el Dockerfile	Al intentar publicar, la plataforma respondió 402: los Spaces con Docker ya requieren suscripción PRO	Descartado (hallazgo)
Render (plan gratuito)	PaaS	Despliegue desde GitHub muy sencillo	512 MB de RAM; la API necesita 1.8 GB	Descartado

El plan original era publicar una demo gratuita en Hugging Face Spaces. Al ejecutar el script de publicación, la plataforma rechazó la creación del Space con el código 402 (pago requerido): durante el proyecto cambió sus condiciones y los Spaces con Docker pasaron a ser de pago. Como el Tema 16 indica que el despliegue real en la nube es demostrativo y no obligatorio, decidí no contratar ningún servicio. En su lugar hice la simulación que propone el tutorial del tema: ejecutar sola la imagen que se desplegaría en el PaaS y probarla igual que en la nube (apartado 8.4, paso 4). El hallazgo muestra que la elección de la plataforma depende también de costos y condiciones que cambian, por lo que el diseño no debe atarse a un solo proveedor.

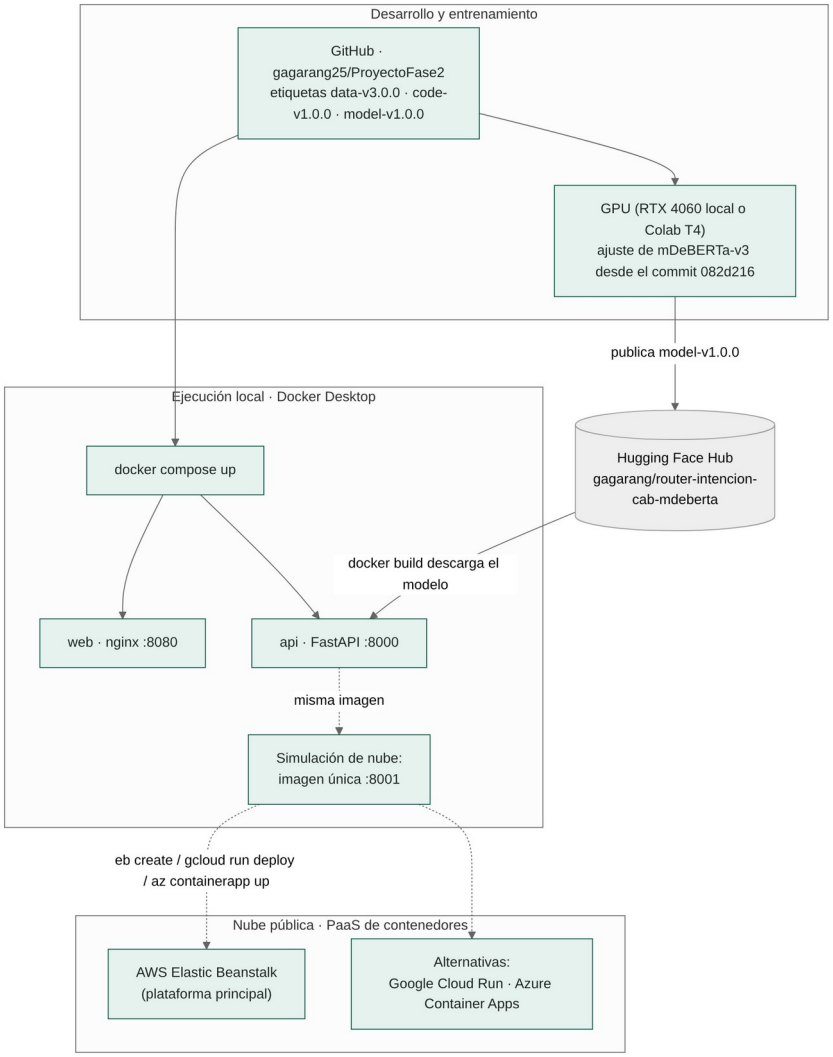


Figura 2: Flujo de despliegue: código, modelo, ejecución local y nube.

8.2 Requerimientos técnicos

Tabla 6: Requerimientos por escenario (medidos en la ejecución local).

Elemento	Ejecución local	Nube (PaaS)
Hardware	CPU de 4 núcleos, 8 GB de RAM (al menos 4 GB asignados a Docker), 10 GB libres en disco	2 vCPU y 4 GB de RAM por instancia (por ejemplo t3.medium en AWS); sin GPU
Software	Docker Desktop 4.x con WSL 2 (Windows) o Docker Engine 24+ con Compose v2; Git; Python 3.10+ solo para las pruebas funcionales	EB CLI (AWS), gcloud (Google) o az (Azure)
Cuentas	GitHub (código) y Hugging Face (descarga pública del modelo)	Cuenta del proveedor de nube
Red	Puertos 8080 (chat) y 8000 (API) libres; internet para construir la imagen	HTTPS público que asigna la plataforma
Imágenes	API: 1.51 GB de contenido (4.37 GB en disco una vez descomprimida); web: 21 MB (73.7 MB en disco)	Solo la imagen de la API
Consumo	API: 1.7 a 1.9 GB de RAM en reposo; web: 15 MB	Igual

La configuración se hace con variables de entorno, para que la misma imagen funcione en cualquier entorno sin cambiar código. En local se leen del archivo .env; en la nube se capturan en la consola de la plataforma.

Tabla 7: Variables de entorno y argumentos de construcción.

Variable	Tipo	Valor por defecto	Para qué sirve
MODELO_ROUTER	build arg	gagarang/router-intencion-cab-mdeberta	Repositorio del router en Hugging Face Hub
MODELO_REVISION	build arg	model-v1.0.0	Etiqueta del modelo que se descarga
GIT_COMMIT	build arg	desconocido	Commit que se despliega (se ve en /api/salud)
FECHA_REFERENCIA	entorno	2026-10-28	Fecha "de hoy" del SAP simulado
UMBRAL_CONFIANZA	entorno	el de config_cab.json (0.90)	Umbral del router
HILOS_CPU	entorno	2	Hilos de CPU para el modelo de lenguaje
CORS_ORIGENES	entorno	localhost:8080	Dominios que pueden llamar a la API
BITACORA_RUTA	entorno	/app/bitacora/ conversaciones.jsonl	Archivo de la bitácora
PORT	entorno	8000	Puerto en el que escucha la API

8.3 Contenerización con Docker

Un contenedor empaqueta la aplicación con sus dependencias y su configuración para que se comporte igual en cualquier computadora (Susnjara y Smalley, s.f.; Red Hat, 2026). El Dockerfile es la receta, la imagen es el resultado de seguir la receta y el contenedor es la imagen en ejecución (Banerjee, 2024). El repositorio tiene dos Dockerfile y un docker-compose.yml (Tabla 8).

Tabla 8: Archivos de contenerización.

Archivo	Imagen	Contenido	Dónde se usa
Dockerfile (raíz)	cab-chatbot-api:1.0.0	Python 3.12 slim, PyTorch para CPU, dependencias con versión exacta, router descargado de Hugging Face, modelos de ETA y riesgo, SAP simulado, pruebas y el chat estático	Servicio api en local y única imagen en la nube
frontend/Dockerfile	cab-chatbot-web:1.0.0	nginx con el chat y el proxy inverso hacia la API	Servicio web en local
docker-compose.yml	—	Levanta web y api, abre el chat solo cuando la API está sana y monta la carpeta de la bitácora	Ejecución local

El Dockerfile de la API sigue las fases de dockerización del Tema 15 (Banerjee, 2024). Cada decisión tiene un propósito:

- **Imagen base python:3.12-slim:** ligera y con la versión de Python con la que se probaron las librerías.
- **PyTorch para CPU desde su índice oficial:** evita descargar las librerías de GPU, que en el servidor no se usan.
- **Dependencias antes que el código:** Docker guarda cada paso como una capa; si solo cambia el código, no reinstala las librerías.
- **Versiones exactas en requirements.txt:** los modelos .joblib solo se abren con la versión de scikit-learn con la que se guardaron.
- **Modelo descargado con su etiqueta (model-v1.0.0):** la imagen queda completa y no depende de internet al arrancar.
- **Usuario sin privilegios (uid 1000):** reduce el daño posible si alguien vulnera la aplicación.

- **HEALTHCHECK:** Docker consulta /api/salud cada 30 segundos; docker compose abre el chat solo cuando la API ya cargó los modelos.
- **Variable PORT:** la plataforma en la nube puede elegir el puerto sin cambiar la imagen.

8.4 Pipeline de despliegue paso a paso

Paso 1. Preparación del entorno. Instalar Docker Desktop y Git, clonar el repositorio y crear el archivo de configuración:

```
git clone https://github.com/gagarang25/ProyectoFase2.git
cd ProyectoFase2
copy .env.example .env                  # Git Bash o Linux: cp .env.example .env
```

Paso 2. Implementación del modelo. El router ya está publicado en Hugging Face Hub con la etiqueta model-v1.0.0, y los modelos tabulares están en backend/modelos/. Para entrenarlos de nuevo: ejecutar entrenamiento/entrenar_router.py en una GPU (cuaderno de Colab o tarjeta local) y publicarlo con entrenamiento/publicar_modelo_hf.py; los modelos de ETA y riesgo se entrenan con entrenamiento/entrenar_modelos_entrega.py en CPU. Cada modelo guarda el commit con el que se entrenó.

Paso 3. Configuración de servicios y ejecución local. Con Docker Desktop abierto:

```
docker compose build --build-arg GIT_COMMIT=$(git rev-parse --short HEAD)
docker compose up -d
docker compose ps                       # web y api deben aparecer como (healthy)
docker compose exec api pytest -q       # pruebas automáticas dentro del contenedor
```

La primera construcción tardó 7.8 minutos (467 s), porque descarga PyTorch y el modelo; las siguientes reutilizan las capas guardadas. Al terminar, el chat queda en http://localhost:8080, la documentación de la API en http://localhost:8080/docs y el estado en http://localhost:8000/api/salud. Para detener: docker compose down.

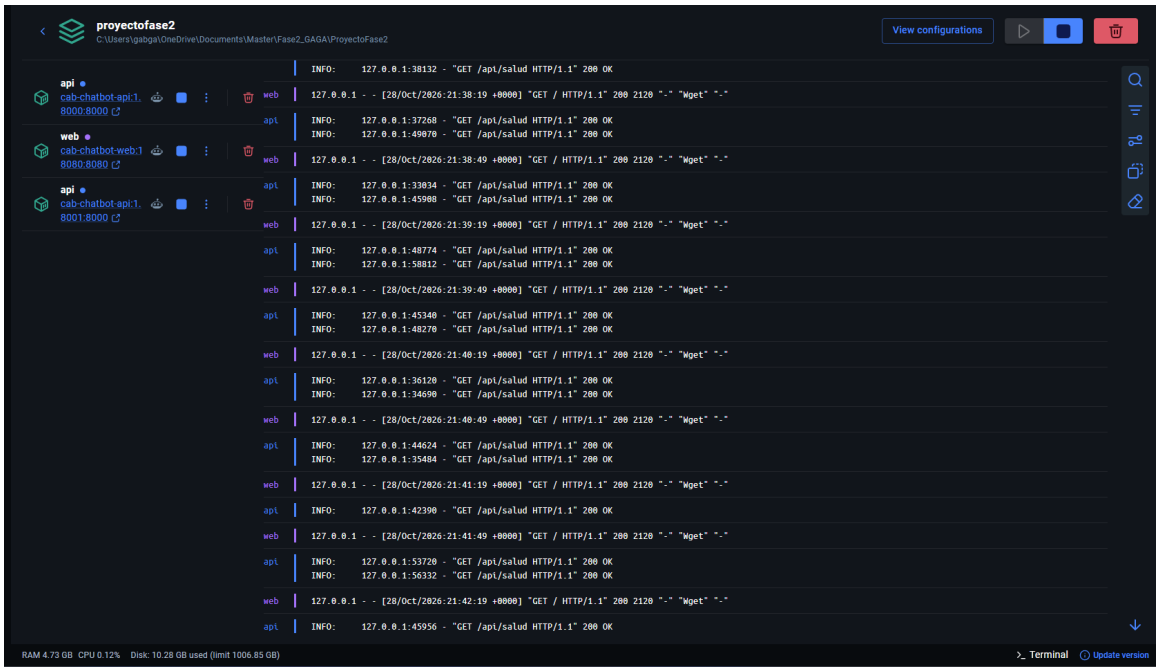


Figura 3: Docker Desktop: api (8000) y web (8080) del compose y la imagen única (8001), con los healthchecks respondiendo 200.

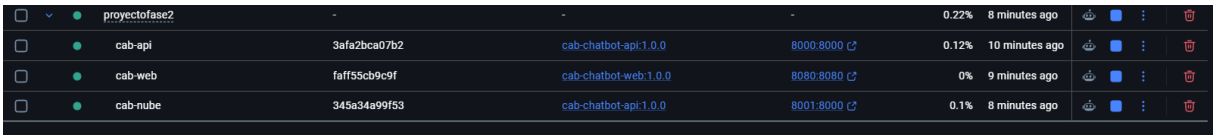


Figura 4: Contenedores en ejecución con su imagen y su puerto: cab-api, cab-web y cab-nube (imagen única).

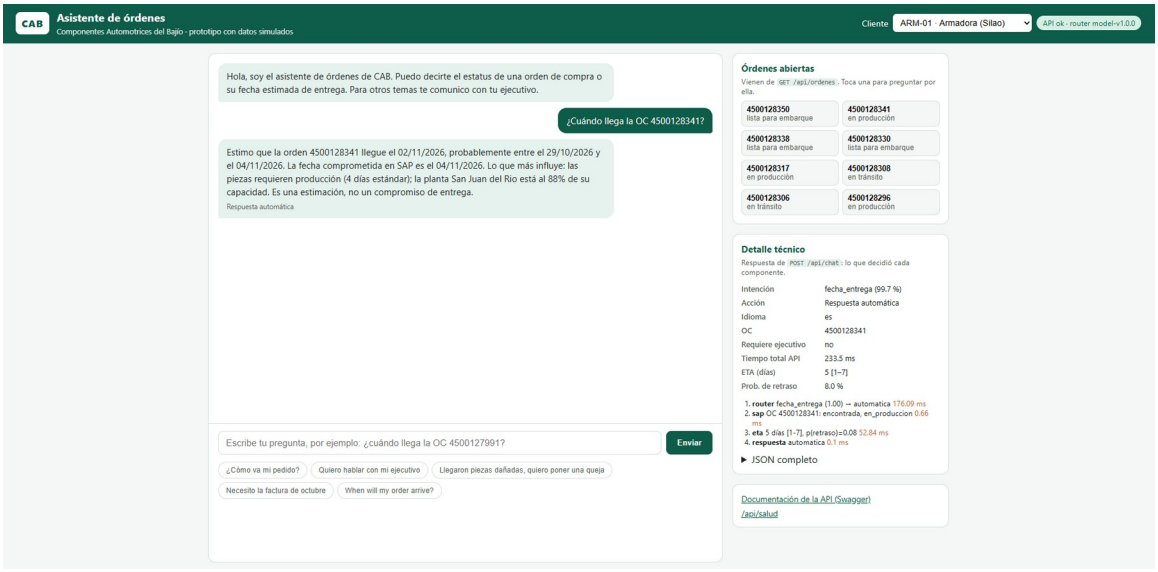


Figura 5: Chat funcionando en http://localhost:8080 con el panel de detalle técnico.



Figura 6: Documentación interactiva de la API (Swagger) en /docs.

Paso 4. Simulación del despliegue en la nube. Como en el tutorial del Tema 16, antes de subir la imagen al PaaS se ejecuta sola, sin nginx ni compose, tal como correría en la nube. La misma API entrega el chat en la ruta raíz:

```
docker run -d --name cab-nube -p 8001:8000 cab-chatbot-api:1.0.0
python pruebas/pruebas_funcionales.py --url http://localhost:8001
```

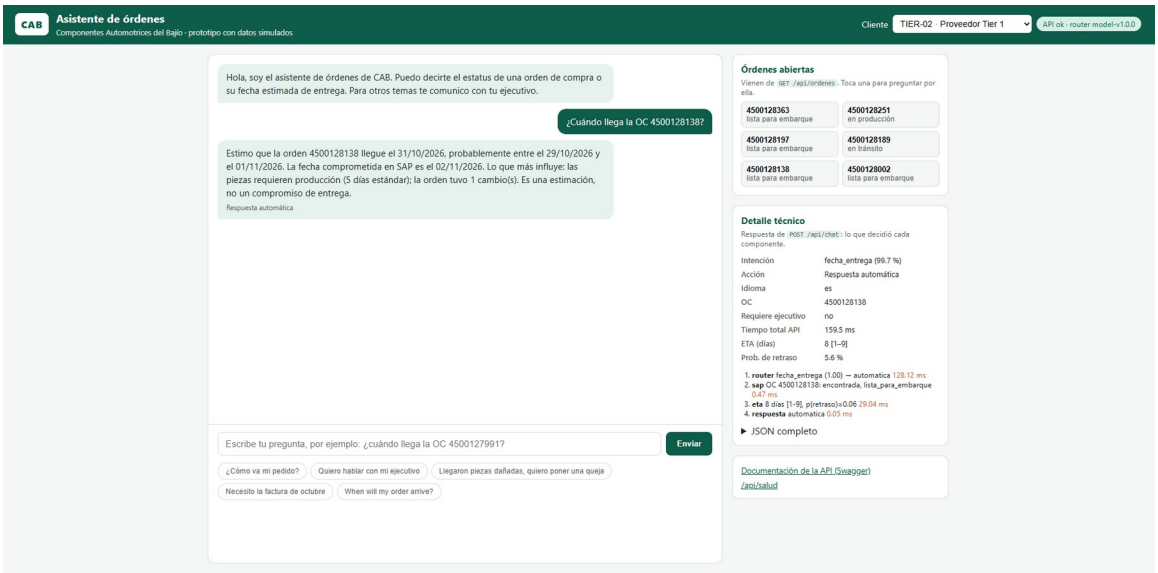


Figura 7: La imagen única sirviendo el chat en el puerto 8001 (simulación del PaaS).

Paso 5. Despliegue en AWS Elastic Beanstalk. Elastic Beanstalk detecta el Dockerfile en la raíz del repositorio, construye la imagen en la instancia y la expone con un dominio propio (Amazon Web Services [AWS], s.f.). Hay que elegir una instancia con al menos 4 GB de memoria, porque la API usa 1.8 GB:

```
pip install awsebcli
eb init -p docker cab-chatbot --region us-east-1
eb create cab-chatbot-env --instance_type t3.medium --single \
  --envvars FECHA_REFERENCIA=2026-10-28,HILOS_CPU=2
eb open                                  # abre la URL pública del entorno
eb terminate cab-chatbot-env             # elimina los recursos al terminar
```

Alternativa: Google Cloud Run. Se publica la imagen en Artifact Registry y Cloud Run la ejecuta con 4 GiB de memoria y 2 vCPU (Google Cloud, s.f.):

```
docker tag cab-chatbot-api:1.0.0 us-central1-docker.pkg.dev/PROYECTO/cab/api:1.0.0
docker push us-central1-docker.pkg.dev/PROYECTO/cab/api:1.0.0
gcloud run deploy cab-chatbot --image us-central1-docker.pkg.dev/PROYECTO/cab/api:1.0.0 \
  --region us-central1 --memory 4Gi --cpu 2 --port 8000 --allow-unauthenticated
```

Alternativa: Azure Container Apps. Es la opción natural para la nube híbrida si CAB usa servicios de Microsoft, porque puede conectarse a una red virtual privada (Microsoft, s.f.):

```
az containerapp up --name cab-chatbot --resource-group rg-cab --location eastus \
  --image <registro>/cab-chatbot-api:1.0.0 --target-port 8000 --ingress external
```

8.5 Validación posterior al despliegue

Después de cada despliegue se ejecuta la misma lista de verificación, basada en el checklist de validación local del Tema 15. El script pruebas/pruebas_funcionales.py automatiza los puntos 2 a 7 y solo usa la librería estándar de Python, así que funciona contra cualquier URL.

Tabla 9: Lista de verificación posterior al despliegue y resultado obtenido.

#	Verificación	Cómo se comprueba	Resultado
1	Los contenedores están sanos	docker compose ps	api y web (healthy)
2	La API responde y el router cargó	GET /api/salud	estado ok; router cargado en 3.1 s
3	Se despliega la versión correcta	git_commit y router.version en /api/salud	70fa349 y model-v1.0.0 (entrenado en 082d216)

#	Verificación	Cómo se comprueba	Resultado
4	El chat responde estatus y fecha	Casos F04 a F15	Pasan
5	No hay fuga entre clientes	Casos E01 a E03	OC ajena = no encontrada / 404
6	Las entradas inválidas se rechazan	Casos E04 a E09 y E13	401, 405 o 422
7	El modelo del contenedor es el validado	160 mensajes de la prueba externa	160/160 respuestas iguales al entrenamiento
8	El tiempo de respuesta es aceptable	100 solicitudes a /api/chat	Mediana 163 ms, p95 188 ms

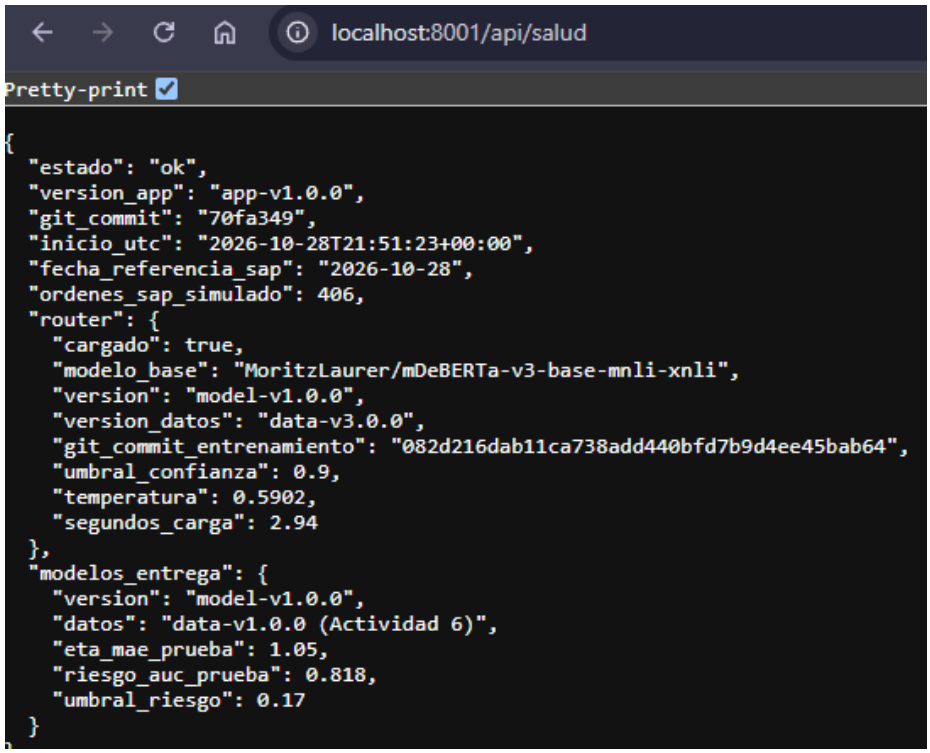


Figura 8: Respuesta de /api/salud en la imagen única: commit desplegado, versión del modelo y commit de entrenamiento.

En la ejecución local pasaron 35 de 35 casos funcionales y extremos, y 35 de 35 con la imagen única. El detalle está en el documento de validación y pruebas.

9. Operación, monitoreo y mantenimiento

9.1 Monitoreo

- **Disponibilidad y latencia:** el HEALTHCHECK de Docker o de la plataforma consulta /api/salud; cada respuesta incluye el encabezado X-Tiempo-ms.
- **Uso del chatbot:** la bitácora JSON permite calcular los indicadores de la Fase 1: porcentaje de consultas resueltas sin ejecutivo, mensajes no entendidos por idioma (R02) y escalamientos.
- **Desempeño de los modelos y calidad de los datos (riesgo R10):** alarmas probadas en el documento de validación: MAE del ETA mayor a 1.3 días, cobertura del rango menor a 85 %, recall del riesgo menor a 0.75 o un índice de estabilidad poblacional (PSI) mayor a 0.2 en las

variables de entrada. Esta última avisa antes de que existan entregas reales con las que medir el error.

9.2 Mantenimiento

- **Preventivo:** reentrenamiento trimestral de los modelos tabulares, actualización mensual de dependencias con la suite de pruebas como red de seguridad, y revisión mensual de la bitácora.
- **Correctivo y reversión (rollback):** cada parte tiene su etiqueta (data-v3.0.0, model-v1.0.0, app-v1.0.0). Para volver atrás se despliega la imagen anterior o se reconstruye con MODELO_REVISION apuntando a la etiqueta previa. En la Fase 3 este proceso se automatizará con integración y despliegue continuos.
- **Gestión de incidentes:** si el router no carga, la API arranca en estado degradado y escala todas las conversaciones a un ejecutivo (falla segura); /api/salud muestra el motivo.

Tabla 10: Problemas frecuentes y cómo resolverlos.

Síntoma	Causa probable	Solución
port is already allocated	Otro programa usa el puerto 8000 u 8080	Cerrar ese programa o cambiar el puerto de la izquierda en docker-compose.yml (por ejemplo 8081:8080)
El contenedor api termina con código 137	Docker no tiene memoria suficiente	Docker Desktop → Settings → Resources: asignar al menos 4 GB
401 o 404 al descargar el modelo en el build	El modelo es privado o el nombre o la etiqueta no existen	Hacer público el modelo o revisar MODELO_ROUTER y MODELO_REVISION
web queda unhealthy aunque el chat abre	En Alpine, localhost resuelve primero a IPv6 y nginx escucha en IPv4 (ocurrió en la prueba)	El HEALTHCHECK usa 127.0.0.1 (corregido en el repositorio)
api queda unhealthy	El modelo no cargó o tarda más que el periodo de arranque	docker compose logs api; /api/salud muestra estado degradado y el motivo
Error de CORS desde otro dominio	El dominio no está en CORS_ORIGENES	Agregar el dominio a la variable y reiniciar
Elastic Beanstalk marca el entorno en rojo	Instancia con poca memoria o puerto distinto	Usar t3.medium o mayor; el Dockerfile expone el puerto 8000

10. Riesgos, limitaciones y supuestos

- **Datos simulados:** las órdenes y los mensajes de CAB son simulados, y la prueba externa la escribió el autor. Los resultados deben confirmarse con el piloto de 5 clientes de la Fase 1.
- **Degradación de los modelos:** los escenarios de drift muestran que una planta saturada sube el error del ETA a 1.34 días. Por eso existen las alarmas del apartado 9.1.
- **ETA desde la fecha del pedido:** el modelo no se actualiza con eventos de rastreo del transportista; si la orden ya superó la fecha estimada, la API lo indica y marca riesgo alto en lugar de dar una fecha pasada.
- **Supuestos operativos:** autenticación con tokens de demostración, una sola instancia y la fecha de referencia fija del SAP simulado.

11. Consideraciones éticas y de uso responsable

- **Supervisión humana:** el chatbot no decide nada por el cliente; ante duda, queja o pedido urgente con riesgo, pasa a un ejecutivo (riesgo R03).

- **Veracidad:** el sistema no redacta fechas; las respuestas aclaran que la fecha es una estimación y no un compromiso.
- **Sesgo:** el router se probó por idioma (español 0.956, inglés 0.975 de exactitud) y el modelo de ETA por transportista y región; las rutas con poco historial tienen mayor error y rango más amplio (riesgo R01).
- **Privacidad:** la bitácora no guarda el texto de los mensajes y cada cliente solo ve sus órdenes.
- **Transparencia:** el chat muestra la intención, la confianza y los motivos de la estimación.

12. Trazabilidad y versionado

Tabla 11: Versiones de los elementos del sistema.

Elemento	Versión	Dónde se registra
Datos de entregas	data-v1.0.0 (A6)	datos/entregas/versiones.json (SHA-256)
Mensajes del router	data-v3.0.0	datos/mensajes_cab/versiones.json; etiqueta de Git en 082d216
Código	code-v1.0.0	Etiqueta de Git en 082d216
Router mDeBERTa-v3	model-v1.0.0	Hugging Face Hub; config_cab.json con commit, datos, temperatura y umbral
Modelos de ETA y riesgo	model-v1.0.0	backend/modelos/metadatos_modelos.json; etiqueta de Git en eb707ef
Aplicación	app-v1.0.0	Etiqueta de Git; /api/salud muestra la versión y el commit desplegado (70fa349 en la validación)
Documento	1.0	docs/ del repositorio

13. Glosario de términos

- **API REST:** conjunto de direcciones (endpoints) con las que un programa pide datos o acciones a otro por HTTP.
- **Contenedor:** programa empaquetado con todo lo que necesita para funcionar igual en cualquier computadora.
- **Imagen:** plantilla de solo lectura a partir de la cual se crean los contenedores.
- **Inferencia:** uso de un modelo ya entrenado para producir una predicción.
- **PaaS:** plataforma como servicio: el proveedor administra servidores y escalado; uno entrega la aplicación.
- **Router de intención:** modelo que decide qué quiere el cliente para enviar el mensaje al componente correcto.
- **Umbral de confianza:** probabilidad mínima para que el chatbot responda solo; por debajo pregunta o escala.
- **Drift:** cambio en los datos o en el mundo que hace que un modelo pierda precisión con el tiempo.

14. Anexos técnicos

- Anexo A. Diagramas de arquitectura y de despliegue (Figuras 1 y 2; fuente en docs/diagramas/).
- Anexo B. Especificación de la API: http://localhost:8000/docs y /openapi.json.

- Anexo C. Configuración: Dockerfile, frontend/Dockerfile, frontend/nginx.conf, docker-compose.yml y .env.example.
- Anexo D. Evidencias de ejecución: docs/evidencias/ (construcción, estado, pytest, pruebas funcionales, bitácora).

Uso de inteligencia artificial generativa en la documentación

El Tema 16 propone usar Microsoft Copilot para generar la documentación técnica a partir de una plantilla alineada con ISO/IEC 23053. Este manual sigue la plantilla generada con Copilot que se compartió en el tema, y para redactar usé Claude (Anthropic) con el mismo propósito: estructurar el contenido, generar borradores de guías paso a paso y listas de verificación, y adaptar el lenguaje a un lector técnico que no conoce el proyecto. La IA no sustituyó la verificación: cada comando se ejecutó, cada cifra salió de los archivos de resultados y cada referencia se abrió para confirmarla. La Tabla 12 resume los prompts principales.

Tabla 12: Prompts usados para documentar y qué se verificó.

Prompt (resumen)	Qué aportó la IA	Qué verifiqué o cambié
Actúa como arquitecto de soluciones. Con la plantilla adjunta (ISO/IEC 23053), redacta el manual de despliegue de una API de IA en contenedores para un lector técnico que no conoce el proyecto. Usa solo las cifras de los archivos de resultados.	Contenido de las 14 secciones a partir de la plantilla.	Relacioné cada sección con los riesgos de la Fase 1 y corregí cifras contra los archivos de evidencias.
Compara Elastic Beanstalk, Cloud Run, Azure Container Apps, Hugging Face Spaces y Render para una imagen que necesita 1.8 GB de RAM. No inventes precios.	Tabla de ventajas y limitaciones.	Confirmé en la documentación oficial los comandos y la memoria; agregué el hallazgo del error 402 de Hugging Face.
Explica cada instrucción del Dockerfile en una frase para alguien que solo conoce lo básico de Docker.	Viñetas del apartado 8.3.	Revisé que cada explicación correspondiera al Dockerfile real.
Genera una tabla de solución de problemas de docker compose en Windows.	Lista inicial de problemas.	Dejé los que aplican y agregué el del healthcheck de IPv6, que sí ocurrió.

Conclusión

Desplegar un modelo resultó ser tanto un problema de ingeniería como de ciencia de datos. El router que en la A4 funcionaba dentro de un cuaderno tuvo que convertirse en un archivo versionado, con su umbral y el commit con el que se entrenó, que una imagen de Docker descarga siempre igual. La prueba de que funcionó es que el modelo dentro del contenedor dio exactamente las mismas 160 respuestas que en el entrenamiento. La elección de mDeBERTa-v3 también definió la estrategia: por la memoria que necesita descarté los planes gratuitos y elegí un PaaS de contenedores, y el cambio de condiciones de Hugging Face confirmó que conviene no depender de un solo proveedor. Con la misma imagen la solución corre en una laptop y, con los comandos de este manual, en Elastic Beanstalk, Cloud Run o Azure. Ese es el valor de la contenerización: que el modelo se comporte igual en cualquier lugar.

Referencias

Amazon Web Services. (s.f.). *QuickStart: Deploy a Docker application to Elastic Beanstalk*. AWS Elastic Beanstalk Developer Guide. Recuperado el 28 de octubre de 2026 de https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/docker-quickstart.html

Banerjee, I. (2024). *Dockerizing your machine learning model: A step-by-step guide*. Medium. https://medium.com/@indradumnabanerjee/dockerizing-your-machine-learning-model-a-step-by-step-guide-12e92f8c960e

Cámara de Diputados del H. Congreso de la Unión. (2025). *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación, 20 de marzo de 2025. https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf

Chen, M. (2025). *¿Qué es una API (interfaz de programación de aplicaciones)?* Oracle. https://www.oracle.com/latam/cloud/cloud-native/api-management/what-is-api/

China, C., y Goodwin, M. (s.f.). *IaaS, PaaS, SaaS: ¿cuál es la diferencia?* IBM. https://www.ibm.com/mx-es/think/topics/iaas-paas-saas

Google Cloud. (s.f.). *Configure memory limits for services*. Cloud Run documentation. Recuperado el 28 de octubre de 2026 de https://docs.cloud.google.com/run/docs/configuring/memory-limits

He, P., Gao, J., y Chen, W. (2023). DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. En *The Eleventh International Conference on Learning Representations (ICLR 2023)*. https://arxiv.org/abs/2111.09543

International Organization for Standardization. (2022). *ISO/IEC 23053:2022 Framework for artificial intelligence (AI) systems using machine learning (ML)*. https://www.iso.org/standard/74438.html

Laurer, M., van Atteveldt, W., Casas, A., y Welbers, K. (2024). Less annotating, more classifying: Addressing the data scarcity issue of supervised machine learning with deep transfer learning and BERT-NLI. *Political Analysis, 32*(1), 84–100. https://doi.org/10.1017/pan.2023.20

Martínez, M. (2024). *¿Qué es la nube pública, privada e híbrida? ¿Cuáles son sus ventajas y desventajas?* NIC México. https://www.nicmexico.mx/contenido?title=que-es-la-nube-publica-privada-e-hibrida-cuales-son-sus-ventajas-y-desventajas

Microsoft. (s.f.). *Deploy Azure Container Apps with the az containerapp up command*. Microsoft Learn. Recuperado el 28 de octubre de 2026 de https://learn.microsoft.com/en-us/azure/container-apps/containerapp-up

Red Hat. (2026). *Docker: qué es, cómo funciona y sus ventajas*. https://www.redhat.com/es/topics/containers/what-is-docker

SG Systems. (2025). *ISO/IEC 23053 — Marco del ciclo de vida del sistema de IA*. https://sgsystemsglobal.com/es/glosario/Marco-del-ciclo-de-vida-del-sistema-de-IA-seg%C3%BAn-ISO-IEC-23053/

Susnjara, S., y Smalley, I. (s.f.). *¿Qué son los contenedores?* IBM. https://www.ibm.com/mx-es/think/topics/containers

The Postman Team. (2023). *What are the components of an API?* Postman Blog. https://blog.postman.com/what-are-the-components-of-an-api/